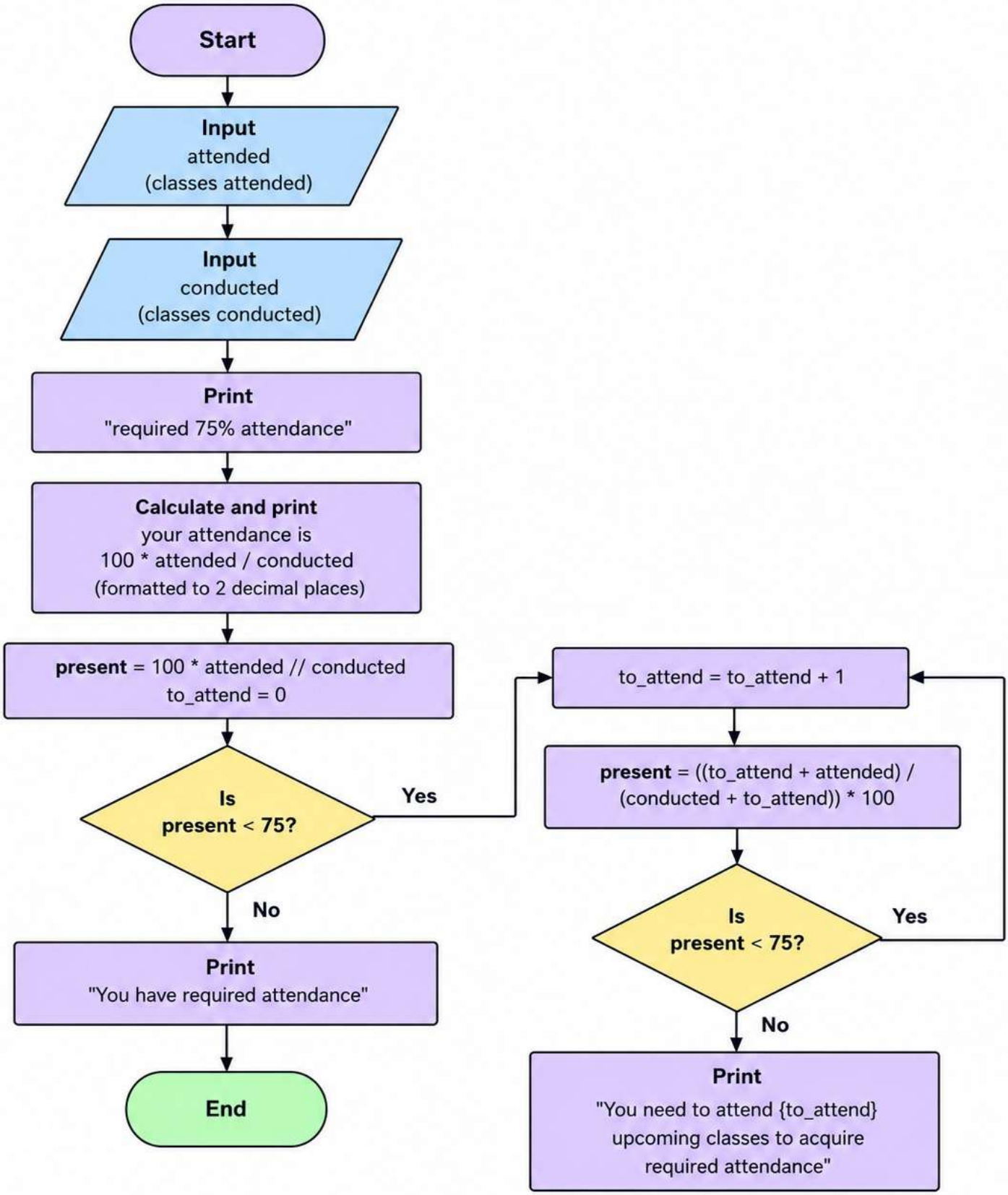
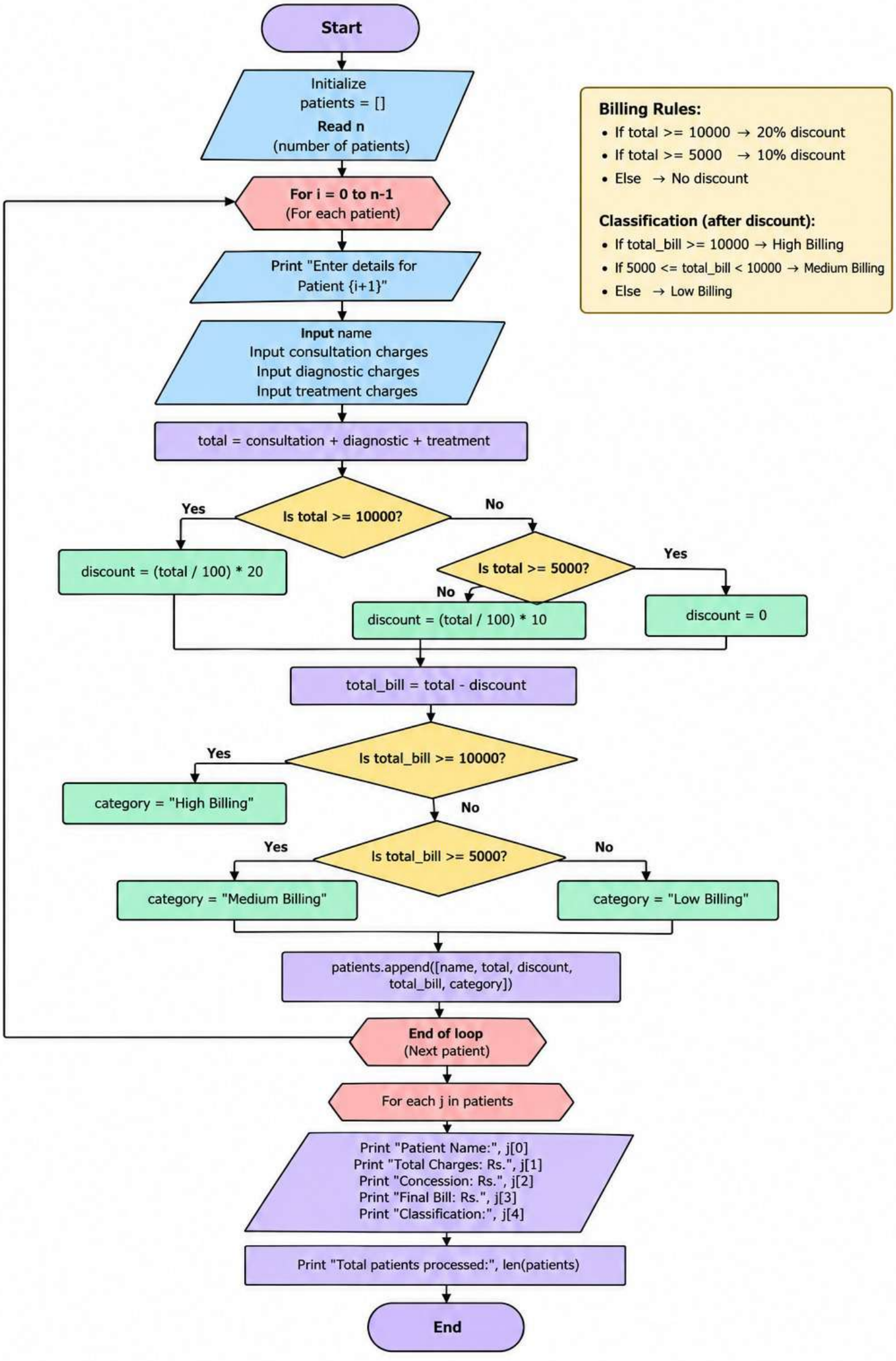
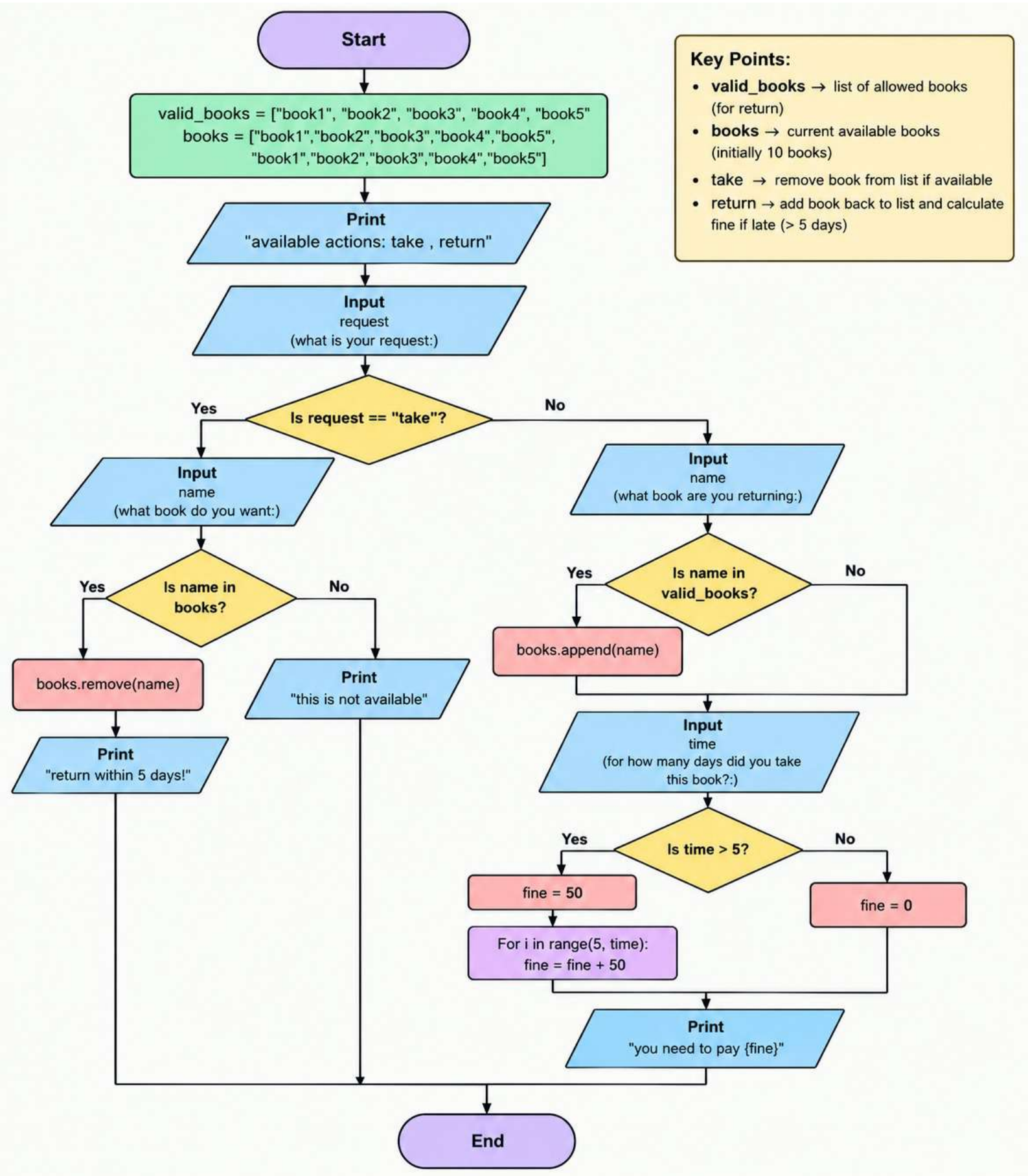
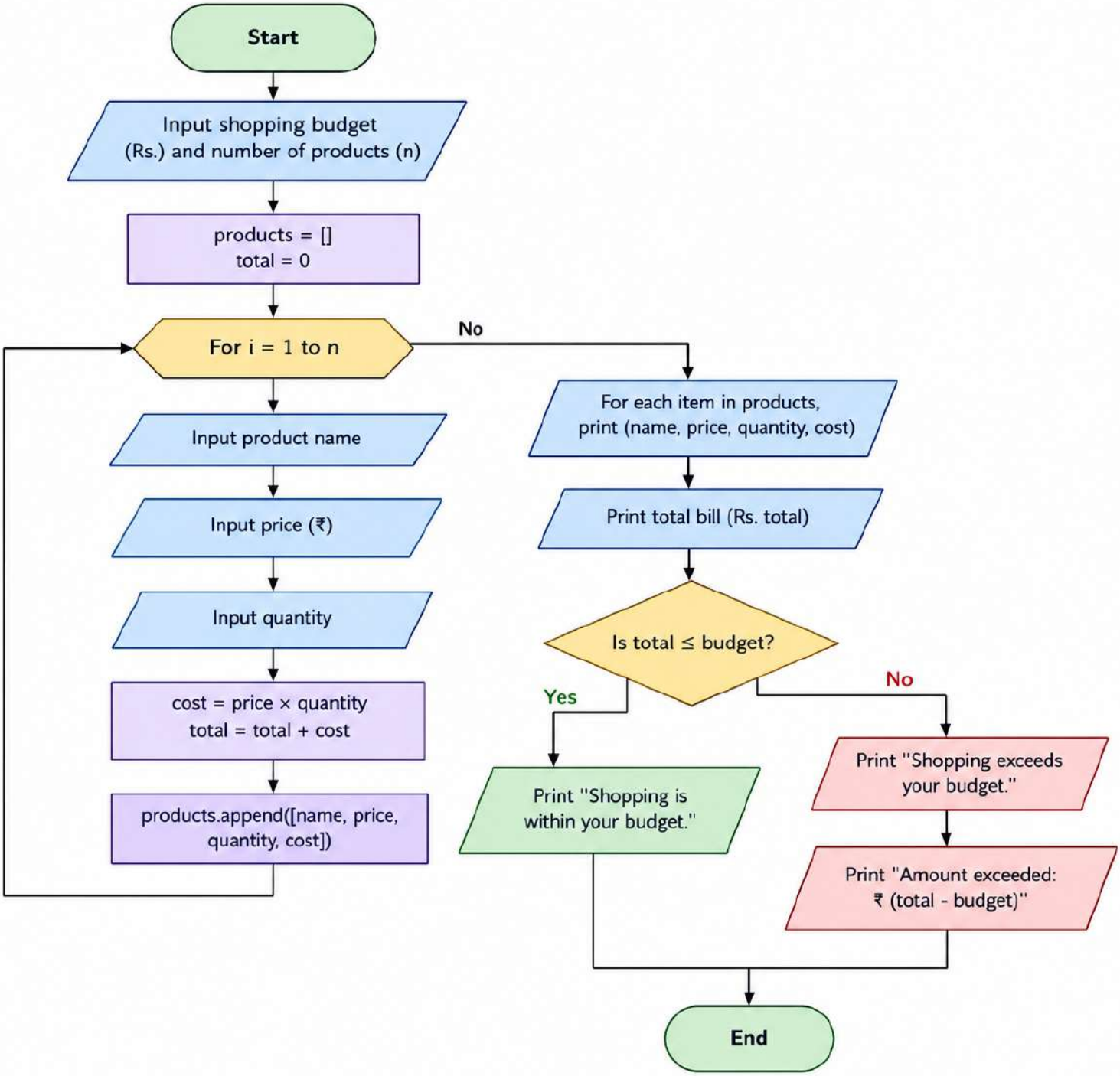
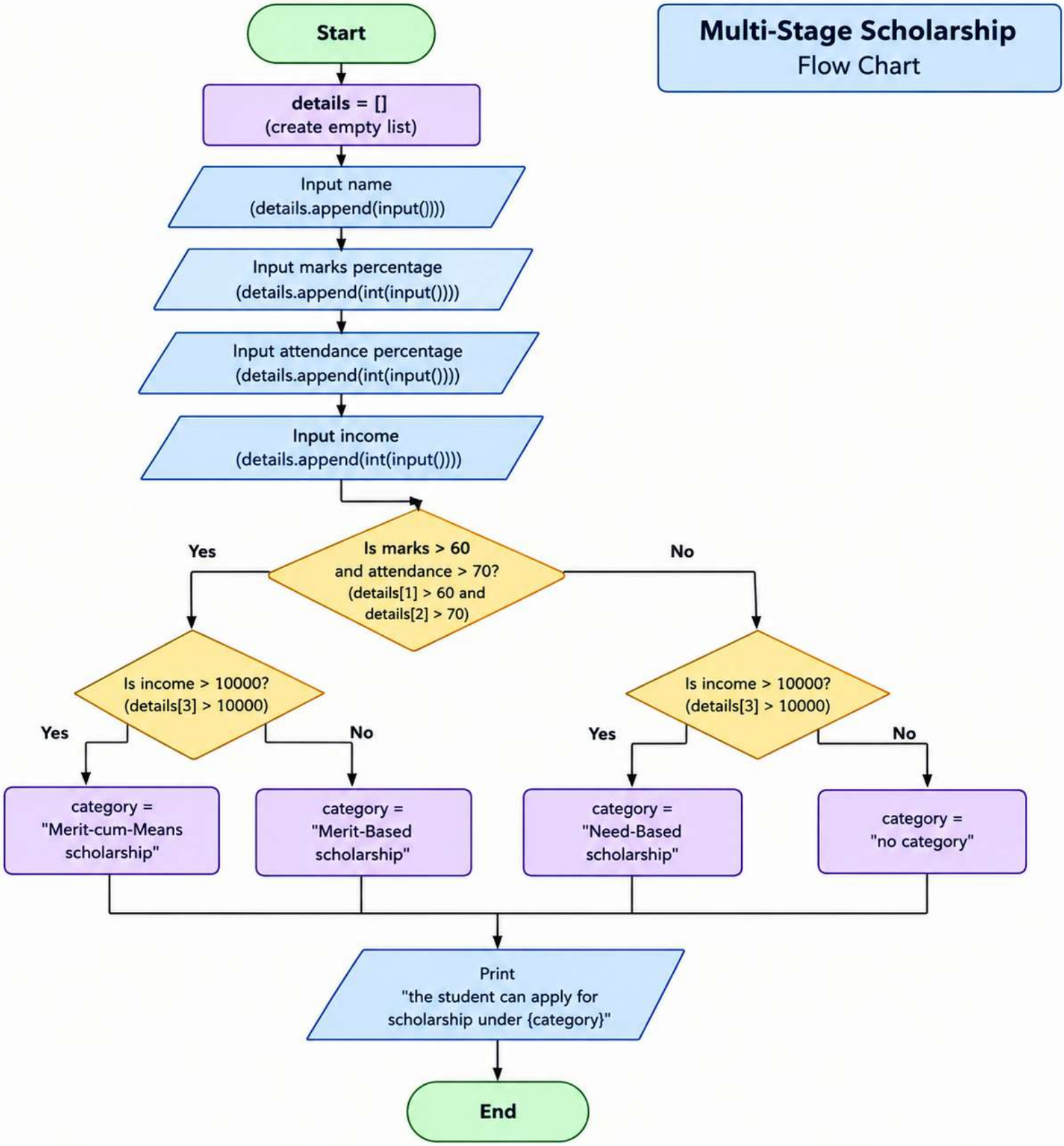
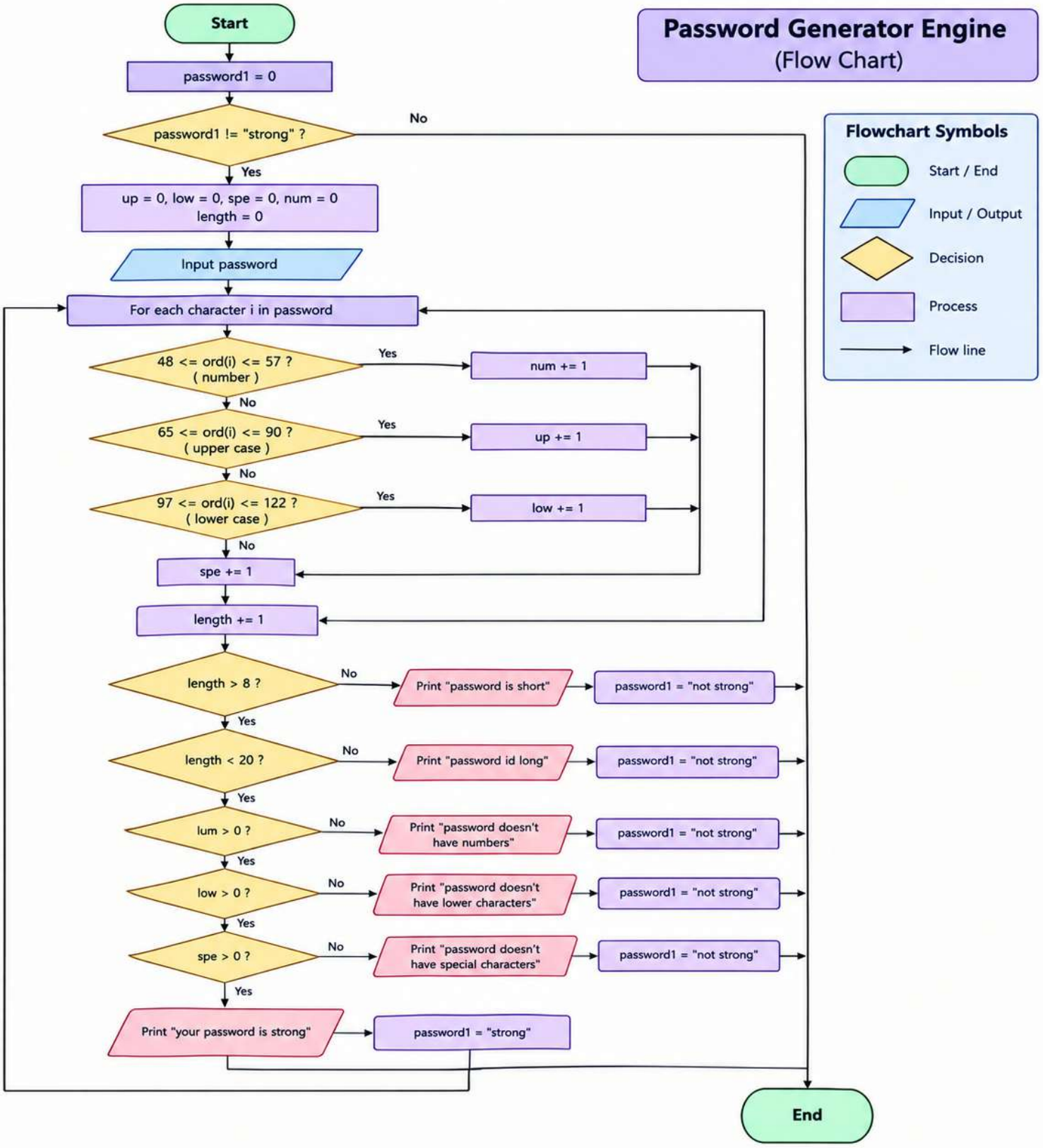

Start
Input
attended
(classes attended)
Input
conducted
(classes conducted)
Print
"required 75% attendance"
Calculate and print
your attendance is
100 * attended / conducted
(formatted to 2 decimal places)
present = 100 * attended // conducted
to_attend = 0
Is
present < 75?
Yes
No
Print
"You have required attendance"
End
to_attend = to_attend + 1
present = ((to_attend + attended) /
(conducted + to_attend)) * 100
Is
present < 75?
Yes
No
Print
"You need to attend {to_attend}
upcoming classes to acquire
required attendance"

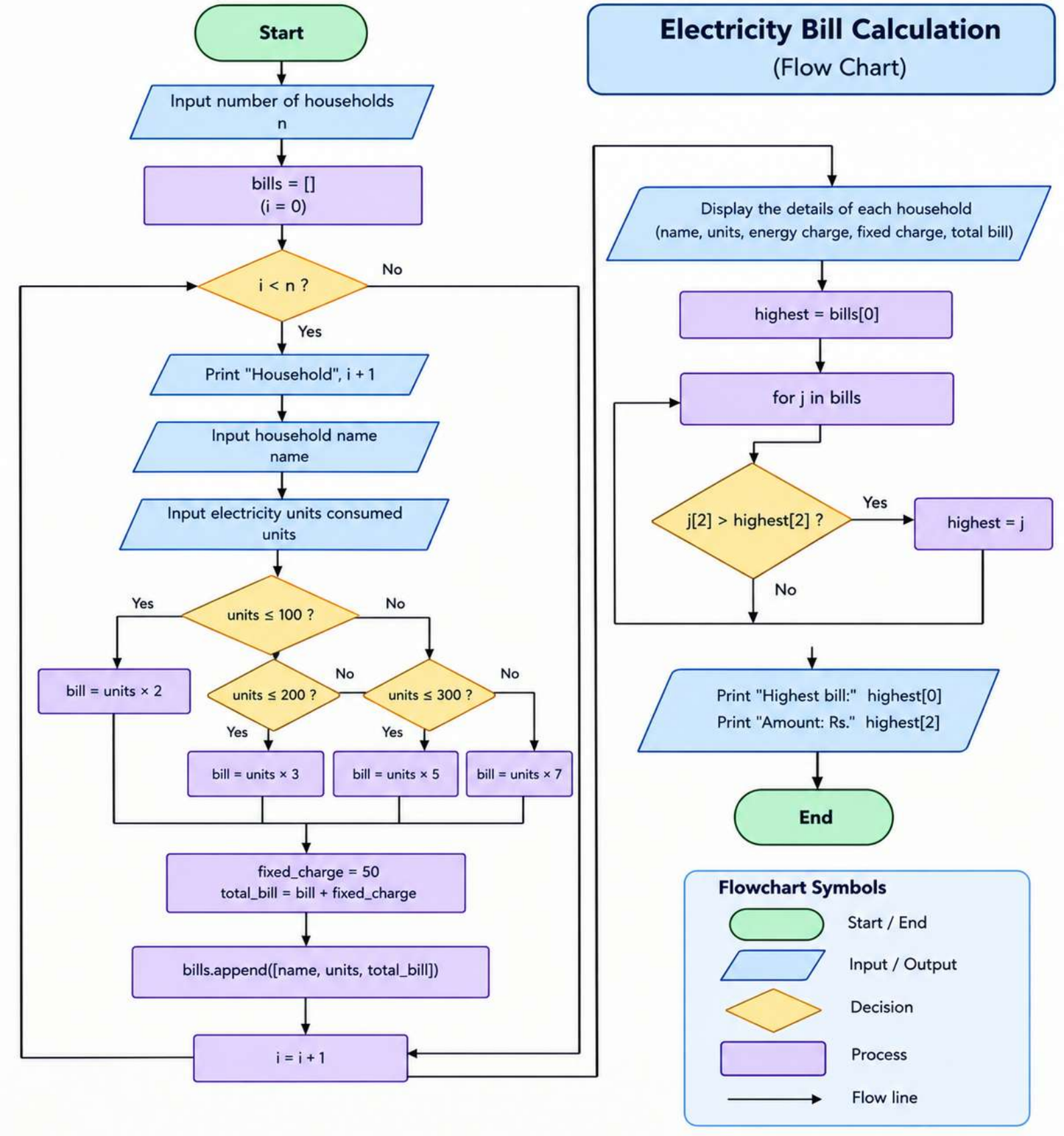

Start
Initialize
patients = []
Read n
(number of patients)
For i = 0 to n-1
(For each patient)
Print "Enter details for
Patient {i+1}"
Input name
Input consultation charges
Input diagnostic charges
Input treatment charges
total = consultation + diagnostic + treatment
Is total >= 10000?
Yes
No
discount = (total / 100) * 20
Is total >= 5000?
Yes
No
discount = (total / 100) * 10
discount = 0
total_bill = total - discount
Is total_bill >= 10000?
Yes
No
category = "High Billing"
Is total_bill >= 5000?
Yes
No
category = "Medium Billing"
category = "Low Billing"
patients.append([name, total, discount,
total_bill, category])
End of loop
(Next patient)
For each j in patients
Print "Patient Name:", j[0]
Print "Total Charges: Rs.", j[1]
Print "Concession: Rs.", j[2]
Print "Final Bill: Rs.", j[3]
Print "Classification:", j[4]
Print "Total patients processed:", len(patients)
End
Billing Rules:
If total >= 10000 → 20% discount
If total >= 5000 → 10% discount
Else → No discount
Classification (after discount):
If total_bill >= 10000 → High Billing
If 5000 <= total_bill < 10000 → Medium Billing
Else → Low Billing

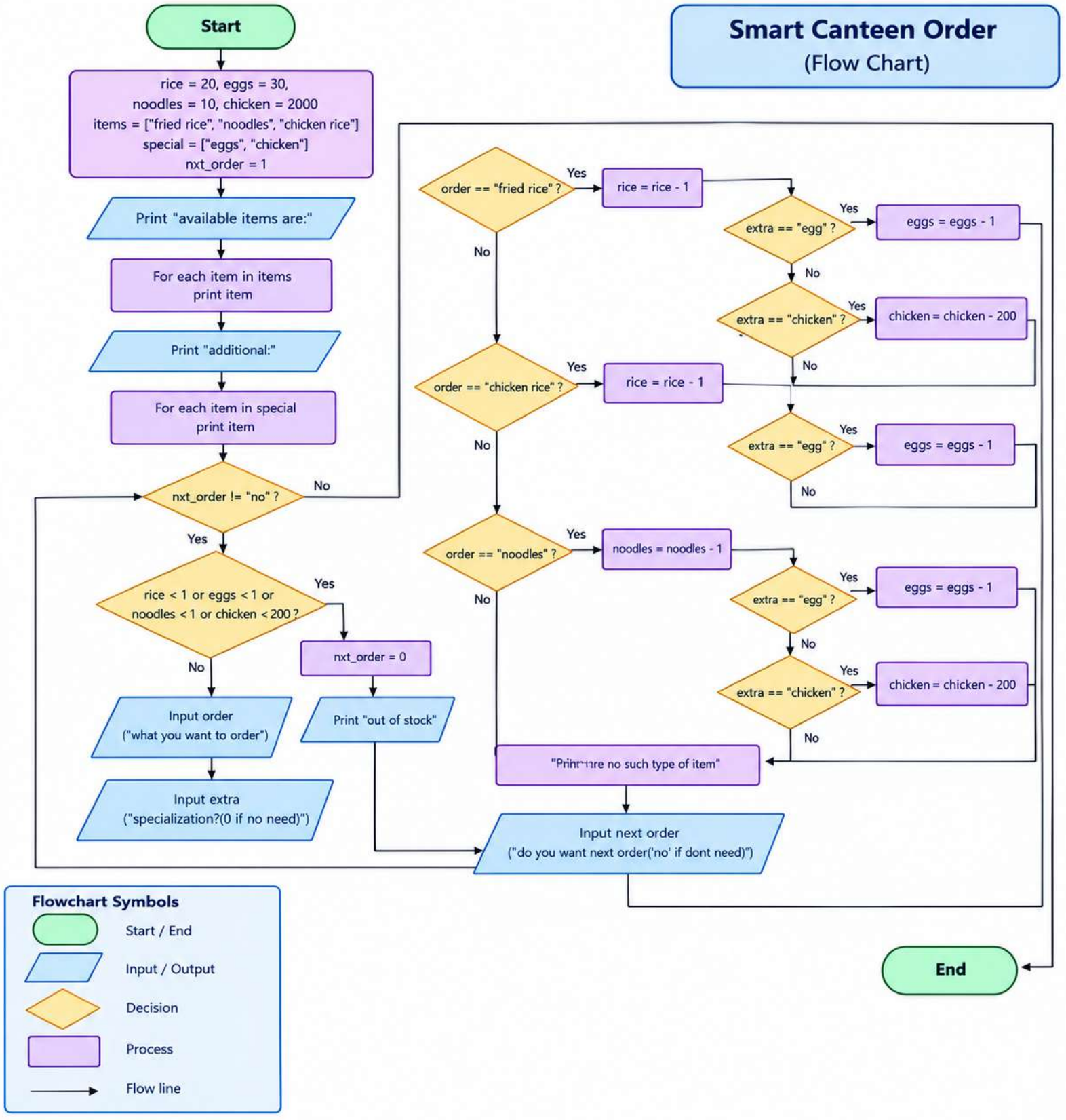

Start
valid_books = ["book1", "book2", "book3", "book4", "book5"
books = ["book1","book2","book3","book4","book5",
"book1","book2","book3","book4","book5"]
Print
"available actions: take , return"
Input
request
(what is your request:)
Is request == "take"?
Yes
No
Input
name
(what book do you want:)
Is name in books?
Yes
No
books.remove(name)
Print
"return within 5 days!"
Print
"this is not available"
Input
name
(what book are you returning:)
Is name in valid_books?
Yes
No
books.append(name)
Input
time
(for how many days did you take this book?:)
Is time > 5?
Yes
No
fine = 50
For i in range(5, time):
fine = fine + 50
fine = 0
Print
"you need to pay {fine}"
End
Key Points:
valid_books → list of allowed books (for return)
books → current available books (initially 10 books)
take → remove book from list if available
return → add book back to list and calculate fine if late (> 5 days)

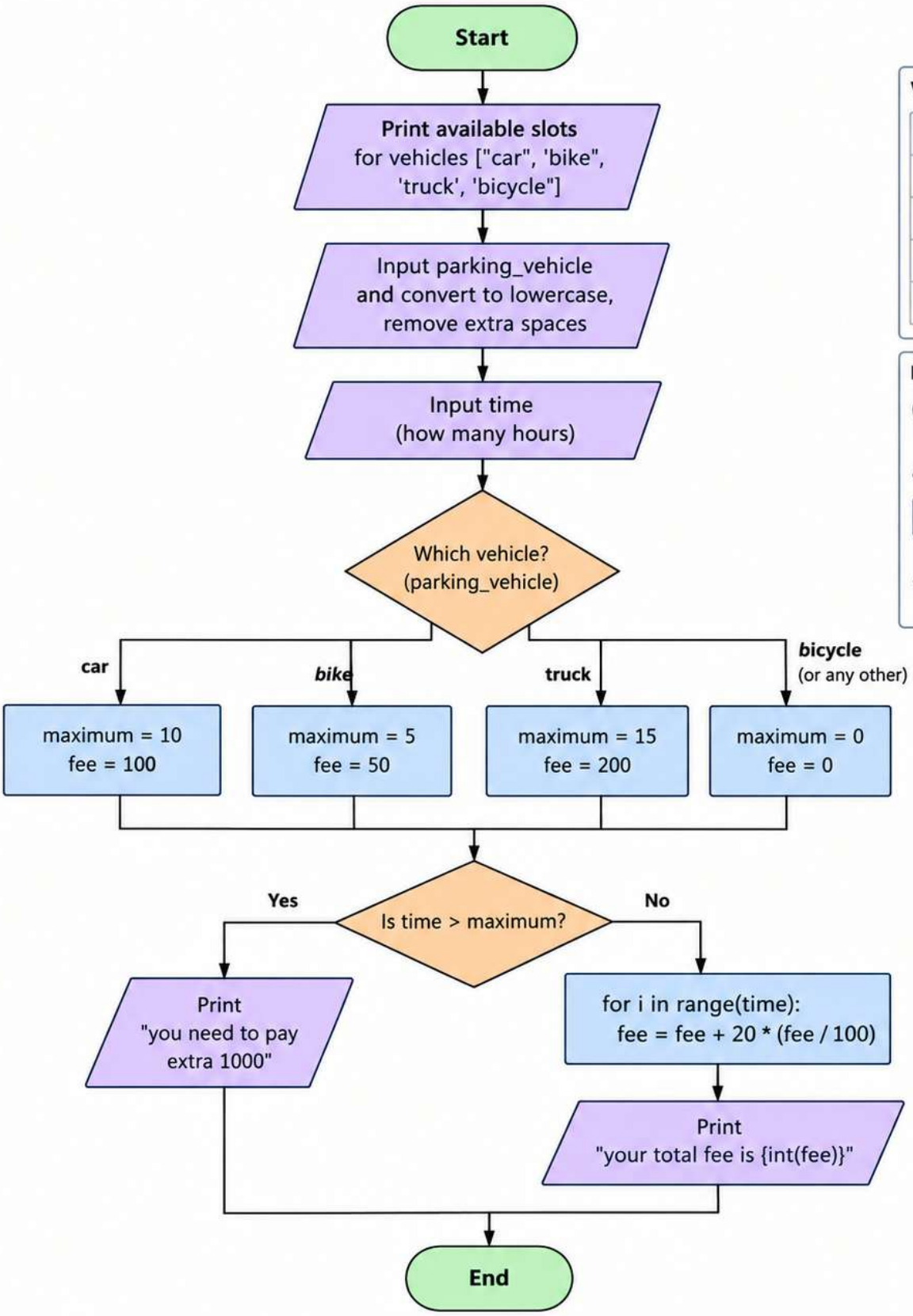

Start
Input shopping budget (Rs.) and number of products (n)
products = []
total = 0
For i = 1 to n
No
Input product name
Input price (₹)
Input quantity
cost = price × quantity
total = total + cost
products.append([name, price, quantity, cost])
For each item in products, print (name, price, quantity, cost)
Print total bill (Rs. total)
Is total ≤ budget?
Yes
No
Print "Shopping is within your budget."
Print "Shopping exceeds your budget."
Print "Amount exceeded: ₹ (total - budget)"
End


Multi-Stage Scholarship
Flow Chart
Start
details = []
(create empty list)
Input name
(details.append(input()))
Input marks percentage
(details.append(int(input())))
Input attendance percentage
(details.append(int(input())))
Input income
(details.append(int(input())))
Is marks > 60
and attendance > 70?
(details[1] > 60 and
details[2] > 70)
Yes
No
Is income > 10000?
(details[3] > 10000)
Yes
No
Is income > 10000?
(details[3] > 10000)
Yes
No
category =
"Merit-cum-Means
scholarship"
category =
"Merit-Based
scholarship"
category =
"Need-Based
scholarship"
category =
"no category"
Print
"the student can apply for
scholarship under {category}"
End


Password Generator Engine
(Flow Chart)
Flowchart Symbols
Start / End
Input / Output
Decision
Process
Flow line
Start
password1 = 0
password1 != "strong" ?
No
Yes
up = 0, low = 0, spe = 0, num = 0
length = 0
Input password
For each character i in password
48 <= ord(i) <= 57 ?
(number)
Yes
num += 1
No
65 <= ord(i) <= 90 ?
(upper case)
Yes
up += 1
No
97 <= ord(i) <= 122 ?
(lower case)
Yes
low += 1
No
spe += 1
length += 1
length > 8 ?
No
Print "password is short"
password1 = "not strong"
Yes
length < 20 ?
No
Print "password id long"
password1 = "not strong"
Yes
lum > 0 ?
No
Print "password doesn't have numbers"
password1 = "not strong"
Yes
low > 0 ?
No
Print "password doesn't have lower characters"
password1 = "not strong"
Yes
spe > 0 ?
No
Print "password doesn't have special characters"
password1 = "not strong"
Yes
Print "your password is strong"
password1 = "strong"
End


Electricity Bill Calculation
(Flow Chart)
Start
Input number of households
n
bills = []
(i = 0)
i < n ?
No
Yes
Print "Household", i + 1
Input household name
name
Input electricity units consumed
units
units ≤ 100 ?
Yes
No
bill = units × 2
units ≤ 200 ?
No
Yes
bill = units × 3
units ≤ 300 ?
No
Yes
bill = units × 5
bill = units × 7
fixed_charge = 50
total_bill = bill + fixed_charge
bills.append([name, units, total_bill])
i = i + 1
Display the details of each household
(name, units, energy charge, fixed charge, total bill)
highest = bills[0]
for j in bills
j[2] > highest[2] ?
Yes
highest = j
No
Print "Highest bill:" highest[0]
Print "Amount: Rs." highest[2]
End
Flowchart Symbols
Start / End
Input / Output
Decision
Process
Flow line


Smart Canteen Order
(Flow Chart)
Start
rice = 20, eggs = 30,
noodles = 10, chicken = 2000
items = ["fried rice", "noodles", "chicken rice"]
special = ["eggs", "chicken"]
nxt_order = 1
Print "available items are:"
For each item in items
print item
Print "additional:"
For each item in special
print item
nxt_order != "no" ?
No
Yes
rice < 1 or eggs < 1 or
noodles < 1 or chicken < 200 ?
Yes
No
nxt_order = 0
Print "out of stock"
Input order
("what you want to order")
Input extra
("specialization?(0 if no need)")
order == "fried rice" ?
Yes
rice = rice - 1
extra == "egg" ?
Yes
eggs = eggs - 1
No
extra == "chicken" ?
Yes
chicken = chicken - 200
No
No
order == "chicken rice" ?
Yes
rice = rice - 1
extra == "egg" ?
Yes
eggs = eggs - 1
No
No
order == "noodles" ?
Yes
noodles = noodles - 1
extra == "egg" ?
Yes
eggs = eggs - 1
No
No
extra == "chicken" ?
Yes
chicken = chicken - 200
No
No
"Prin re no such type of item"
Input next order
("do you want next order('no' if dont need)")
End
Flowchart Symbols
Start / End
Input / Output
Decision
Process
Flow line

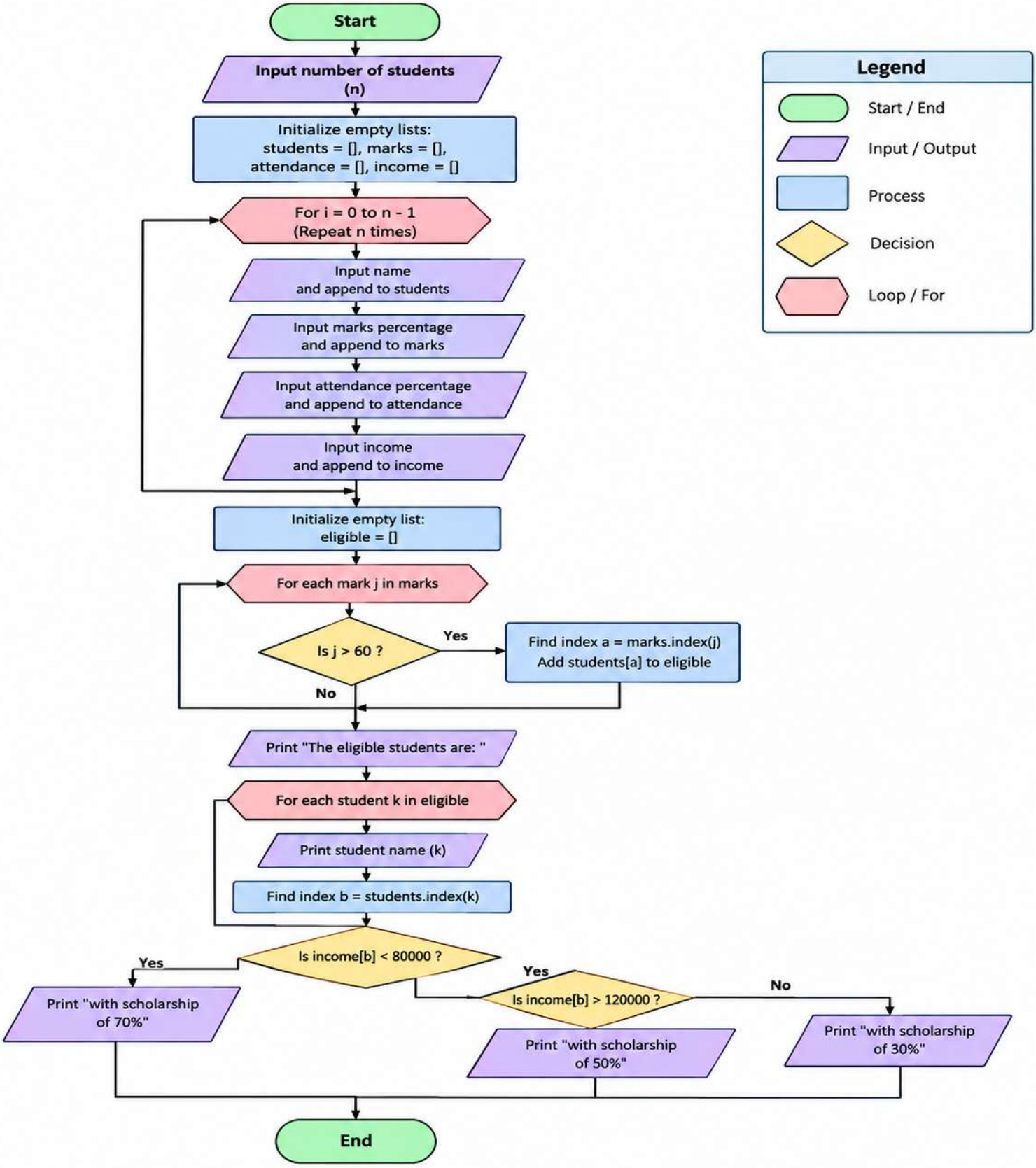


Vehicle Details

Vehicle	Maximum Hours	Base Fee
Car	10	100
Bike	5	50
Truck	15	200
Bicycle	0	0

Legend

- Start / End
- Input / Output
- Process
- Decision


Start
Input number of students (n)
Initialize empty lists: students = [], marks = [], attendance = [], income = []
For i = 0 to n - 1 (Repeat n times)
Input name and append to students
Input marks percentage and append to marks
Input attendance percentage and append to attendance
Input income and append to income
Initialize empty list: eligible = []
For each mark j in marks
Is j > 60 ?
Yes
Find index a = marks.index(j) Add students[a] to eligible
No
Print "The eligible students are: "
For each student k in eligible
Print student name (k)
Find index b = students.index(k)
Is income[b] < 80000 ?
Yes
Print "with scholarship of 70%"
Yes
Is income[b] > 120000 ?
Print "with scholarship of 50%"
No
Print "with scholarship of 30%"
End
Legend
Start / End
Input / Output
Process
Decision
Loop / For